

Guida pratica: lavorare su DEVELOP e aggiornare MAIN solo dopo test

Questa guida ti permette di:

- sviluppare senza toccare main
- testare finche' tutto e' stabile
- promuovere su main solo versioni verificate

1) Regola base

- main = solo produzione stabile
- develop = integrazione delle modifiche testate
- feature/* = lavoro per singola attivita'

2) Setup iniziale (una volta sola)

Verifica branch disponibili:

```
git branch -a
```

Aggiorna i riferimenti remoti:

```
git fetch --all --prune
```

Passa a develop:

```
git checkout develop  
git pull
```

3) Inizio nuova attivita'

Crea un branch dedicato partendo da develop:

```
git checkout develop
git pull
git checkout -b feature/nome-attivita
```

Esempio nome branch:

- feature/fix-scroll-mobile
- feature/sync-csv-public
- feature/fix-bordero-export

4) Sviluppo e commit progressivi

Durante il lavoro:

```
git status
git add -A
git commit -m "feat: descrizione breve"
```

Push del branch feature:

```
git push -u origin feature/nome-attivita
```

5) Test prima della promozione

5.1 Test automatico di integrita'

Nel workspace e' disponibile il task Test System Integrity, oppure da terminale:

```
node test-system.js
```

5.2 Test manuale funzionale (obbligatorio per questo progetto)

Avvio server locale:

```
node unified-server.js
```

Controlli minimi:

- desktop: apri index.html e verifica rendering CSV
- mobile: apri public/mobile1.html e verifica card e scorrimento
- NextCoreo: verifica visualizzazione valore A1 da NextCoreo.csv
- fullscreen: verifica riavvio scorrimento senza salti

Nota progetto:

- se modifichi display.csv o NextCoreo.csv, aggiorna anche la copia in public

6) Aggiornare develop con modifiche testate

Quando la feature e' pronta:

```
git checkout develop
git pull
git merge --no-ff feature/nome-attivita
git push
```

Poi puoi eliminare il branch feature:

```
git branch -d feature/nome-attivita
git push origin --delete feature/nome-attivita
```

7) Promozione da develop a main (solo se stabile)

Opzione consigliata: Pull Request

1. Apri PR da develop verso main su GitHub
2. Controlla diff e file inclusi
3. Verifica checklist test
4. Esegui merge solo se tutto e' verde

Opzione da terminale (se lavori da solo)

```
git checkout main  
git pull  
git merge --no-ff develop  
git push
```

8) Protezione di main su GitHub (fortemente consigliata)

In GitHub:

- Settings
- Branches
- Add branch protection rule su main

Impostazioni consigliate:

- Require a pull request before merging
- Require approvals (almeno 1)
- Dismiss stale approvals when new commits are pushed
- Require status checks to pass (se presenti)
- Include administrators
- Restrict who can push to matching branches

Con queste regole, il push diretto su main viene bloccato.

9) Routine giornaliera consigliata

All'inizio:

```
git checkout develop  
git pull  
git checkout -b feature/nome-task
```

Durante il lavoro:

```
git add -A  
git commit -m "fix: ..."  
git push
```

A fine task:

```
node test-system.js
```

Se i test sono ok:

- merge su develop
- nuova verifica rapida
- PR develop -> main

10) Errori comuni e recupero veloce

Hai committato per errore su main:

```
git checkout -b hotfix/sposta-da-main  
git checkout develop  
git merge --no-ff hotfix/sposta-da-main  
git push
```

Hai file temporanei Excel nello staging:

```
git restore --staged -- "Excel/~$*.xlsm"
```

Hai aggiunto una repo annidata per errore:

```
git rm --cached -r -f VSC_Live_Server
```

11) Checklist finale prima del merge su main

- test-system.js passato
- verifica manuale desktop/mobile ok
- nessun file temporaneo o lock file
- CSV root e public allineati
- commit con messaggi chiari
- diff della PR revisionato

Se tutti i punti sono verificati, la versione e' pronta per main.